

Seminar: Image Segmentation Based on Visual Prompt

Group MNIST

VNUHCM - University of Science

January 18, 2026



23122008 - Mai Duc Minh Huy

23122024 - Chung Tin Dat

Table of contents

1 Introduction

- General Background
- Motivation
- Problem Statement

2 Related Works

- Traditional Methods
- Deep Learning Methods

3 Methodology

- Data Collection
- Model Architecture

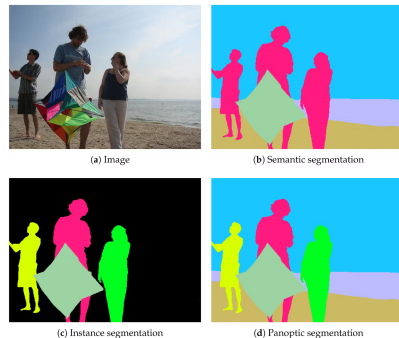
4 Conclusion

5 Discussion



General Background

Image segmentation is a fundamental task in computer vision that involves partitioning a digital image into multiple segments or sets of pixels, often corresponding to different objects or parts of objects. The goal is to simplify or change the representation of an image into something that is more meaningful and easier to analyze.



General Background

Visual prompts are user-provided graphical inputs—such as clicks, boxes, or lines that "tell" a computer vision algorithm where to focus or what specific object to process.

- **Points (Clicks):** The user clicks on a specific object (positive click) or background (negative click).
- **Bounding Boxes:** The user drags a rectangle around the object. This gives the algorithm a clear size and location limit.
- **Scribbles/Strokes:** The user draws a rough line over the object.

video & prompts in one or multiple frames



box

(skipped)

points

mask

The history of this field can be broadly divided into two distinct eras.

- **The Pre-Deep Learning Era: Seed-Based and Energy Models**
- **The Deep Learning Era: From Weak Supervision to Interaction**



The Pre-Deep Learning Era: Seed-Based and Energy Models

Before the rise of neural networks, interactive segmentation relied on mathematical frameworks where user inputs acted as "seeds" or constraints for optimization algorithms. These methods focused on low-level features like color gradients and pixel connectivity:



The Pre-Deep Learning Era: Seed-Based and Energy Models

Before the rise of neural networks, interactive segmentation relied on mathematical frameworks where user inputs acted as "seeds" or constraints for optimization algorithms. These methods focused on low-level features like color gradients and pixel connectivity: Prominent methods from this period include:

- Watershed
- Active Contour
- Grab Cut
- Region Growing



The Pre-Deep Learning Era: Seed-Based and Energy Models

Before the rise of neural networks, interactive segmentation relied on mathematical frameworks where user inputs acted as "seeds" or constraints for optimization algorithms. These methods focused on low-level features like color gradients and pixel connectivity: Prominent methods from this period include:

- Watershed
- Active Contour
- Grab Cut
- Region Growing

The prompt does not provide "understanding" of the object; it simply tells the math where to start calculating and what color values to look for nearby.



The Deep Learning Era: From Weak Supervision to Interaction

The integration of Convolutional Neural Networks (CNNs) allowed models to learn semantic context, making visual prompts significantly more powerful and robust to noise.



The Deep Learning Era: From Weak Supervision to Interaction

The integration of Convolutional Neural Networks (CNNs) allowed models to learn semantic context, making visual prompts significantly more powerful and robust to noise. Key milestones include:

- Box Sup
- Scribble Sup
- Deep Extreme Cut
- Segment Anything



The drive to advance image segmentation stems from both scientific inquiry and practical necessity.

Scientifically,

- Segmentation addresses a fundamental question in perception: while image classification tells us what is in an image, segmentation answers where it is, down to the pixel level; segmentation helps reveal how an image is organized by separating it into meaningful regions.
- Many methods in computer vision rely on segmented regions as inputs. Examples include object detection, recognition, tracking, 3D reconstruction, and medical image analysis. Better segmentation improves the performance of these tasks.

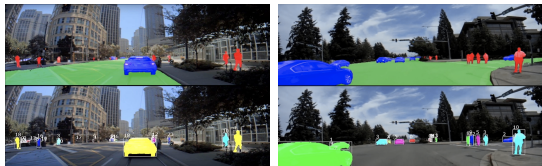


Practically, segmentation is an enabling technology for a multitude of real-world applications.

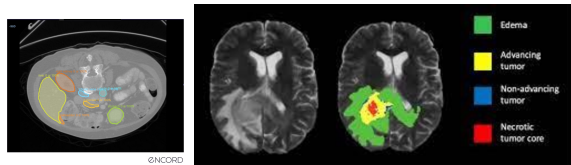


Motivation

Autonomous Driving



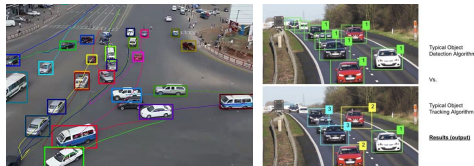
Medical Imaging



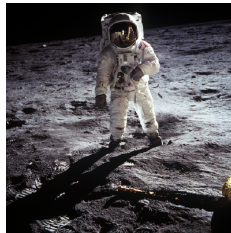
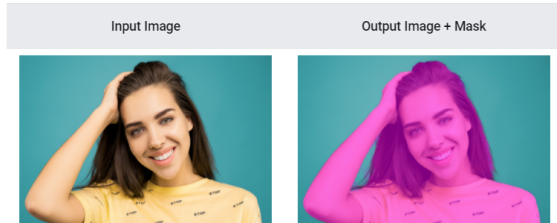
Surveillance



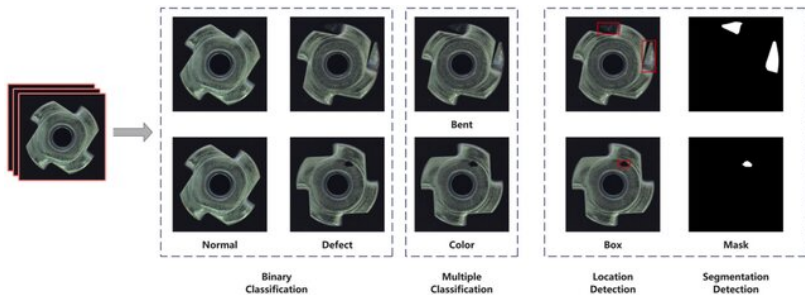
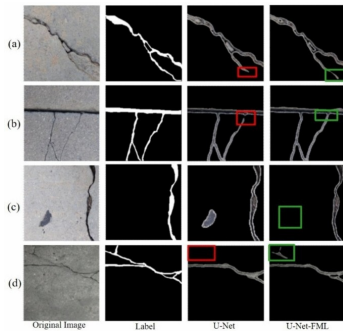
Object Tracking



Digital Photograph



Industrial Inspection



Segment Anything Model (SAM)

“In this work, our goal is to build a foundation model for image segmentation.”



Problem Statement

Image segmentation is the task of grouping pixels that belong to a common semantic object and separating them from other objects and the background.

The core principle of segmentation is guided by two criteria: pixels within the same group should be as similar as possible, while pixels belonging to different groups should be as distinct as possible.

In essence, the task simulates the human cognitive ability to transform raw, unstructured visual data into a structured representation of distinct entities.



Problem Statement

Let $\Omega \subset \mathbb{R}^2$ be the image domain and $I : \Omega \rightarrow \mathbb{R}^k$ the image.

A segmentation is a partition

$$\Omega = \cup_{i=1}^K \Omega_i \quad \Omega_i \cap \Omega_j = \emptyset \text{ for } i \neq j$$

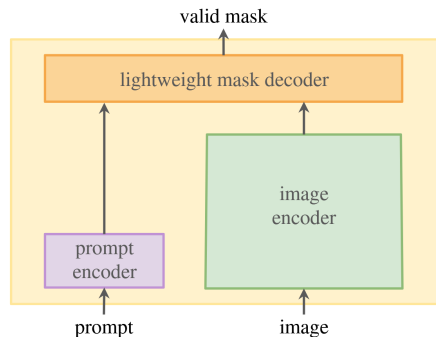
such that pixels within each region share similar properties according to some criterion.



Problem Statement

For a modern promptable segmentation system, the problem can be formally defined by its inputs and outputs.

- **Input:** An image and a "visual prompt" that specifies the object of interest. This prompt can take various forms, such as a point (or multiple points) on the object, a bounding box enclosing it, or even a rough mask.
- **Output:** One or more segmentation masks for the object(s) indicated by the prompt, along with a confidence score for each mask.



Traditional Methods

Methods	Mechanics
Watershed	Flooding process on gradient topography with controlled basins.
Active Contour	Energy minimizing curve driven by internal smoothness and image forces.
Grab Cut	Local similarity expansion based on pixel intensity or color distance.
Region Growing	Probabilistic graph cut with iterative foreground background modeling.



Traditional Methods

Methods	Methodology
	Stage 1: Input preparation
Watershed	Image gradient, foreground and background markers.
Active Contour	Image as continuous energy field, initial contour.
Grab Cut	Image as graph with color distributions (GMMs), bounding box.
Region Growing	Image as pixel grid with feature vectors, seed pixels.



Traditional Methods

Methods	Methodology	
	Stage 2: Propagation/Optimization	Stage 3: Final labeling
Watershed	Flooding until basins meet.	Watershed lines form region boundaries.
Active Contour	Iterative energy minimization via curve evolution.	Final curve represents object boundary.
Grab Cut	Repeated min cut optimization with model updates.	Cut defines segmentation boundary.
Region Growing	Iterative neighbor expansion.	Region boundary from grown region.



Traditional Methods

Methods	Contribution
Watershed	Introduced topographic interpretation of images and marker controlled segmentation to reduce over segmentation.
Active Contour	Formalized segmentation as energy minimization over curves and influenced decades of variational methods.
Grab Cut	Demonstrated practical interactive segmentation with probabilistic modeling.
Region Growing	Established seed based interactive segmentation.



Traditional Methods

Methods	Limitations
Watershed	Sensitive to noise and gradient quality.
Active Contour	Struggles with concave boundaries and weak edges.
Grab Cut	Assumes color separability between foreground and background; fails when distributions overlap.
Region Growing	Highly sensitive to noise and intensity variation, leakage occurs at weak boundaries.



Technique	Mechanics
Box Sup	Unsupervised methods to generates hundreds of potential mask shapes (candidate segments), bounding box is used to filter these candidates.
Scribble Sup	Image is divided into super pixels, use Graph Cuts optimization to label the unmarked pixels.
Deep Extreme Cut	Heatmap from extreme points, CNN extracts feature specifically on the areas highlighted by the heatmap.
Segment Anything	Cross attention between image embedding and prompt embedding.



Technique	Methodology	
	Image representation	Prompt encoding
Box Sup	CNN feature maps extracted from region proposals and full images.	Bounding boxes converted into soft foreground background supervision using objectness and low level segmentation cues.
Scribble Sup	CNN feature maps over full image.	Scribbles transformed into dense label maps via graph based or CRF propagation.
Deep Extreme Cut	CNN encodes image appearance along with spatial context.	Extreme points encoded as distance maps concatenated with image channels.
Segment Anything	Vision transformer generates dense image embeddings.	Prompts encoded into tokens representing points, boxes, or masks

Methods	Methodology
	Mask decoding
Box Sup	CNN trained on pseudo masks predicts dense semantic segmentation masks.
Scribble Sup	CNN outputs semantic masks learned from propagated pseudo labels.
Deep Extreme Cut	Single pass CNN inference produces object mask.
Segment Anything	Transformer based mask decoder predicts multiple candidate masks with confidence scores.



Technique	Contribution
Box Sup	It effectively combined traditional "region proposal" methods (unsupervised) with deep learning (supervised), allowing the network to learn shapes without ever seeing a ground truth mask.
Scribble Sup	It successfully integrated a Graphical Model (Graph Cut) into the deep learning pipeline. The graph propagates the scribble information to nearby pixels based on color/texture, creating a dense mask to train the CNN.
Deep Extreme Cut	It showed that annotating extreme points is 5x faster than drawing polygons but yields results that are nearly as good as fully supervised learning.
Segment Anything	First truly general purpose segmentation foundation model. Enables interactive segmentation, automatic segmentation, and box guided segmentation in one model.



Technique	Limitations
Box Sup	The model is limited by the quality of the candidate generation (MCG/Selective Search). If the proposal algorithm fails to generate a mask that fits the object, the CNN cannot learn it.
Scribble Sup	The "propagation" step relies on traditional color and texture similarity. If the object has low contrast, the graph cut fails to spread the label correctly.
Deep Extreme Cut	The user must provide exactly 4 points, and they must be the extreme edges. It struggles with objects that have disjoint parts because the extreme points might not capture the empty space in between efficiently.
Segment Anything	High computational cost during preprocessing due to large encoder. Not ideal for pixel perfect tasks such as thin structures or high precision medical boundaries without fine tuning. Mask quality depends on prompt placement.



The fundamental challenge in building a foundation model for segmentation is the lack of web-scale segmentation data.

Segmentation masks are not naturally occurring and must be manually created.

Traditional approaches would require paying annotators to label billions of masks-economically infeasible and temporally prohibitive.



SAM Data Engine:

- ① Assisted Manual Annotation (4.3M Masks from 120K Images)
- ② Semi-Automatic Annotation (5.9M Additional Masks, 180K Images)
- ③ Fully Automatic Annotation (1.1B Masks, 11M Images)



1. Assisted Manual Annotation (4.3M Masks from 120K Images):

Workflow:

- ① Professional annotators view an image in a web-based interface.
- ② They click foreground/background points to indicate an object of interest.
- ③ SAM's pre-computed image embedding (from MAE-ViT-H) is queried with these prompts.
- ④ The lightweight Prompt Encoder and Mask Decoder run in real-time ($< 50\text{ms}$), generating an initial mask prediction.
- ⑤ Annotators refine the mask using pixel-precise brush and eraser tools.
- ⑥ The final mask is saved.



Quantitative Dynamics:

- **Initial annotation time:** 34 seconds per mask (comparable to COCO annotation standards).
- **Model capacity progression:** ViT-B $\rightarrow$ ViT-H during this stage (1 retraining cycle per major improvement).
- **Average masks per image:** 20 $\rightarrow$ 44 (increased diversity as model improved).
- **Total masks collected:** 4.3M from 120K images.
- **Retraining cycles:** 6 full model retraining iterations.
- **Total masks collected:** 4.3M from 120K images.



Efficiency Gain Analysis:

The $34 \rightarrow 14$ second progression represents a **$6.5\times$ speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.



2. Semi-Automatic Annotation (5.9M Additional Masks, 180K Images):

Workflow:

- ① Train a generic object detector on all masks from Stage 1. This detector outputs bounding boxes regardless of semantic class (object-agnostic proposal generation).
- ② SAM automatically segments high-confidence regions identified by this detector, generating candidate masks.
- ③ Annotators are presented with images pre-filled with these automatic masks.
- ④ Annotators' task: Label only the *unannotated* objects (those not covered by the automatic masks).
- ⑤ This forces annotators to focus on smaller, more occluded, or semantically ambiguous objects.



Why This Engineering is Critical:

The semi-automatic stage is a **curriculum learning** strategy. By automatically handling easy cases and forcing annotators to label hard cases, the model learns a more balanced representation. This is empirically validated: the model's ability to handle edge cases improved significantly from Stage 1 to Stage 2, as evidenced by the diversity metrics.



Efficiency Gain Analysis:

The $34 \rightarrow 14$ second progression represents a **$6.5\times$ speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.



Why stage 2 is needed?

By automatically handling easy cases and forcing annotators to label hard cases, the model learns a more balanced representation. This is empirically validated: the model's ability to handle edge cases improved significantly from Stage 1 to Stage 2, as evidenced by the diversity metrics.



3. Fully Automatic Annotation (1.1B Masks, 11M Images):

Systematically prompt the model with a regular spatial grid of points (32×32), covering the entire image. Workflow:

- ① Each point is passed as a prompt to the Prompt Encoder.
- ② For each point, SAM predicts **three masks simultaneously**.
- ③ **Theoretical maximum:** $1024 \text{ points} \times 3 \text{ masks per point} = 3072 \text{ masks per image}$.



Filtering, Deduplication, Non-Maximum Suppression

- ① IoU Scoring and Confidence Thresholding
- ② Stability Filtering
- ③ Non-Maximum Suppression (NMS)

IoU Scoring and Confidence Thresholding

- For each of the three masks predicted from a grid point, SAM outputs a **predicted IoU score** (estimated mask quality).
- **Threshold:** Only masks with **predicted IoU > 0.88** are retained.
- **Rationale:** High IoU threshold ensures only high-quality masks enter the deduplication pipeline.
- **Expected reduction:** From 3072 theoretical outputs, this threshold alone eliminates $\sim 80\%$ of predictions (keeping only predictions the model is highly confident about).



Stability Filtering

- A mask is **stable** if small perturbations in the model's confidence threshold (e.g., changing the foreground probability threshold from 0.5 to $0.5 \pm \epsilon$) produce similar masks.
- **Operationally:** Threshold the mask's probability map at both 0.5 and 0.5, and compute the IoU between the two resulting binary masks.
- If this "stability IoU" is high (> 0.95), the mask is kept; otherwise, it is discarded.
- **Why This Matters:** Masks that are brittle (sensitive to small threshold changes) are often segmentation errors or include spurious boundaries. Stability filtering removes low-confidence predictions and keeps robust ones.
- **Further reduction:** $\sim 50\%$ of remaining masks fail stability checks.



Non-Maximum Suppression

- ① Sort masks by predicted IoU score.
- ② Select the highest-scoring mask m_1 .
- ③ For all remaining masks m_i :
 - Compute $\text{IoU}(m_1, m_i) = \frac{|m_1 \cap m_i|}{|m_1 \cup m_i|}$ (Jaccard index).
 - If $\text{IoU}(m_1, m_i) > \text{thresh}_{\text{IoU}}$ (typically 0.7), discard m_i (it is a near-duplicate).
 - If $\text{IoU}(m_1, m_i) \leq \text{thresh}_{\text{IoU}}$, retain m_i (it is a distinct object).
- ④ Repeat on remaining unprocessed masks.



Scale Reduction Summary:

- **Start:** 3072 theoretical masks (32×32 grid $\times$ 3 outputs per grid cell).
- **After IoU filtering (> 0.88):** ~ 600 masks (20% remain).
- **After stability filtering:** ~ 300 masks (50% of IoU-filtered).
- **After NMS ($\text{thresh_IoU} = 0.7$):** ~ 100 masks (final).

Final Calculation:

Total SA-1B Masks = 11M images $\times$ 100 masks/image = 1.1 B masks



Why stage 3 is needed?

A regular grid ensures comprehensive spatial coverage. If an object's boundary passes through or near a grid point, the model will capture it from that point. Objects of varying sizes are covered because the grid is at a fixed, moderate resolution. This is fundamentally different from random point sampling, which can miss objects entirely.



Overall, this three-stage pipeline can be summarized:

- ① **Stage 1** teaches the model to segment *diverse* objects via human guidance.
- ② **Stage 2** forces the model to learn *challenging* objects via curriculum learning.
- ③ **Stage 3** leverages the improved model to generate *scale* without human cost.



Why 3 outputs per prompt?

Empirical analysis during SAM's development showed that image regions typically have at most three levels of semantic hierarchy (whole > part > sub-part). Predicting three masks covers all common ambiguity scenarios without excessive computation.



The Output Token Design:

- The Mask Decoder predicts three masks from a single prompt, ranked by IoU score.
- **Whole object:** The complete, semantically coherent object.
- **Part:** A prominent sub-component (e.g., a wheel on a car, a face on a person).
- **Sub-part:** Fine-grained components (e.g., an eye within a face, a spoke within a wheel).



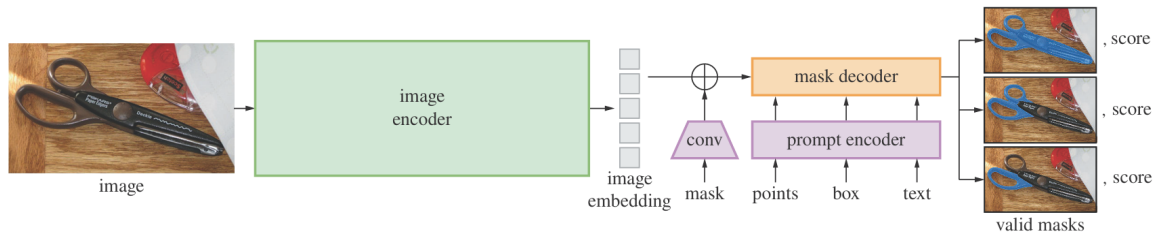
Segment Anything Model

Output token is an embedding vector acts as a query for the final mask prediction (analogous to CLS token in BERT or ViT).

This token gather information about the prompt embeddings and image embeddings to form semantic information required to segment the target object.



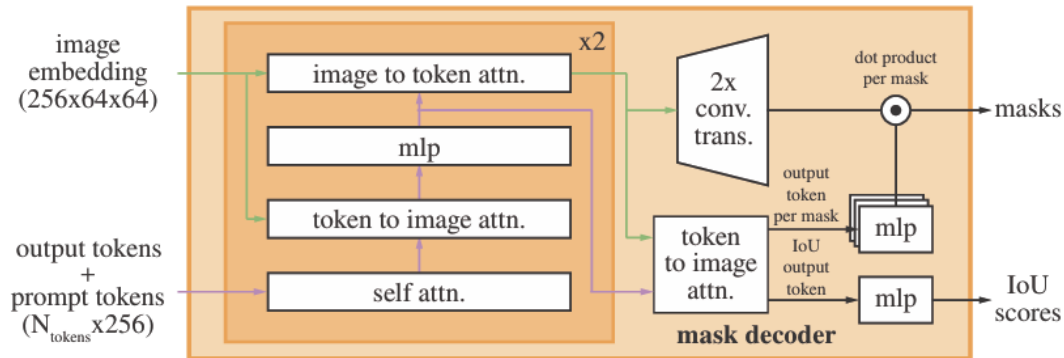
Segment Anything Model



- Image Encoder: MAE pre-trained Vision Transformer (ViT).
- Prompt Encoder:
 - Point, box: Positional encoding summed with learned embeddings for each prompt type.
 - Mask: Convolution and summed element-wise with the image embedding.

Segment Anything Model

- Mask Decoder:



Key Design:

- Image Encoder runs **once per image**, producing a latent embedding.
- This embedding is cached.
- Prompt Encoder and Mask Decoder run **every time the user provides a new prompt** on the cached embedding.
- Result: 50ms per prompt on CPU in a web browser (no GPU required).



Segment Anything Model

In traditional interactive segmentation (e.g., RITM, SimpleClick), the entire network processes the image for each new click, leading to latency. SAM's decoupling means that the first prompt has a cost (cached image encoding) and subsequent prompts are nearly free. For applications with many prompts (e.g., dense instance segmentation with many objects), the amortized cost per object approaches zero.



Segment Anything Model

Image Encoder: A Vision Transformer (ViT) processes images by:

- ① Dividing the image into non-overlapping patches (typically 16×16 pixels).
- ② Flattening each patch into a 1D vector and projecting to embedding dimension $d = 768$ (for ViT-B) or $d = 1280$ (for ViT-H).
- ③ Adding **positional embeddings** to inject spatial information.
- ④ Processing the sequence of patch embeddings through stacked Transformer layers using self-attention.



Segment Anything Model

Prompt Encoder: **Points (Foreground/Background)**:

A point prompt is simply a 2D coordinate (x, y) and a label indicating foreground (+1) or background (-1).

Representation:

- 1 Convert coordinate to positional encoding (using sinusoidal or learned embeddings).
- 2 Embed the foreground/background label using learned embeddings $\mathbf{e}_{\text{fg}}$ and $\mathbf{e}_{\text{bg}}$ (dimension 256).
- 3 Sum the positional and semantic embeddings:

$$\mathbf{p}_{\text{point}} = PE(x, y) + \mathbf{e}_{\text{label}}$$



Segment Anything Model

Prompt Encoder: **Boxes (Bounding Box Prompts)**:

A box is specified by two corners: $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.

Representation:

- 1 Generate two "corner points": $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$.
- 2 Encode each corner with distinct learned embeddings ($\mathbf{e}_{\text{top-left}}$, $\mathbf{e}_{\text{bottom-right}}$).
- 3 Embed the box as:

$$\mathbf{p}_{\text{box}} = [PE(x_{\min}, y_{\min}) + \mathbf{e}_{\text{top-left}}, \quad PE(x_{\max}, y_{\max}) + \mathbf{e}_{\text{bottom-right}}]$$



Segment Anything Model

Prompt Encoder: **Mask (Dense Prompts):**

Representation:

- ① Resize the input mask to match the spatial resolution of the Image Embedding (e.g., 256×256 if image is 1024×1024).
- ② Apply **2D convolutions** to extract spatial features:
 - Input: Binary mask (single channel).
 - Conv layers: 3–4 layers with ReLU activations, stride 1.
 - Output: Feature map of shape $(256 \times 256, C)$ where $C = 256$.
- ③ Element-wise sum with the Image Embedding:

$$\mathbf{z}_{\text{fused}} = \mathbf{z}_{\text{image}} + \mathbf{z}_{\text{mask}}$$



Segment Anything Model

Mask Decoder: Two-Way Transformer Fusion

- 1 **Self-attention on prompts:** Prompts attend to themselves to capture relationships between multiple prompts (e.g., if multiple points are provided, they can "communicate").

$$\mathbf{A}_{\text{prompt}} = \text{Softmax} \left(\frac{\mathbf{Q}_{\text{prompt}} \mathbf{K}_{\text{prompt}}^T}{\sqrt{d}} \right) \mathbf{V}_{\text{prompt}}$$

- 2 **Cross-attention (prompt $\rightarrow$ image):** Prompts query the Image Embedding to extract relevant regions.

$$\mathbf{A}_{\text{p2i}} = \text{Softmax} \left(\frac{\mathbf{Q}_{\text{prompt}} \mathbf{K}_{\text{image}}^T}{\sqrt{d}} \right) \mathbf{V}_{\text{image}}$$



Segment Anything Model

Mask Decoder: Two-Way Transformer Fusion

- 1 **Cross-attention (image $\rightarrow$ prompt):** Conversely, the Image Embedding queries the Prompt Embedding to understand what the user is asking for.

$$\mathbf{A}_{i2p} = \text{Softmax} \left(\frac{\mathbf{Q}_{\text{image}} \mathbf{K}_{\text{prompt}}^T}{\sqrt{d}} \right) \mathbf{V}_{\text{prompt}}$$

- 2 **Update embeddings:** All three attention outputs are summed and passed through layer normalization and MLPs:

$$\mathbf{z}'_{\text{prompt}} = \text{LN}(\mathbf{z}_{\text{prompt}} + \mathbf{A}_{\text{prompt}} + \mathbf{A}_{p2i})$$

$$\mathbf{z}'_{\text{image}} = \text{LN}(\mathbf{z}_{\text{image}} + \mathbf{A}_{i2p})$$



Loss Function

A linear combination of focal loss and dice loss in a 20:1 ratio of focal loss to dice loss.

- Local Loss: Focus to learning energy to hard pixels.

$$FocalLoss = -\sum_{i=1}^n (i - p_i)^{\gamma} \log_b(p_i)$$

- Dice Loss: Differentiable approximation of IoU.

$$DiceLoss = \frac{2 * Intersection}{Prediction + Ground_Truth}$$



Conclusion

This report has surveyed the evolution of image segmentation, tracing its trajectory from early mathematical heuristics to the modern era of foundation models. We began by examining traditional methods such as Watershed and Region Growing, which relied on low-level pixel intensity and required significant user intervention to define boundaries. We then explored the transition to deep learning approaches like BoxSup and DEXTR, which introduced semantic understanding but were often limited by the scale of their training data and specific class vocabularies.



Conclusion

The focal point of our analysis, the Segment Anything Model (SAM), represents a paradigm shift in this domain. By leveraging a massive "Data Engine" and a decoupled architecture—separating a heavy image encoder from a lightweight prompt decoder—SAM has demonstrated that zero-shot generalization is achievable on a scale previously thought impossible. Our review of the methodology confirms that the integration of human-in-the-loop annotation (the "flywheel effect") was accurate in solving the data scarcity problem that hindered previous generations of segmentation models.



Despite the remarkable capabilities of foundation models, our analysis in Chapter 3 highlights inherent limitations regarding computational resources. The reliance on a Vision Transformer (specifically ViT-H) for the image encoder creates a significant bottleneck. While the prompt encoder is efficient (operating in milliseconds), the initial image embedding step is computationally expensive, making the current full-scale model unsuitable for real-time applications on edge devices or mobile platforms without hardware acceleration.



Furthermore, while Chapter 2 highlighted that traditional methods like GrabCut were prone to failure on complex textures, they had the advantage of being lightweight. SAM, conversely, trades computational weight for semantic robustness. Additionally, while the 32×32 grid prompting strategy allows for automatic mask generation, the model still occasionally struggles with highly ambiguous structures or "camouflaged" objects where the boundary between foreground and background is semantically ill-defined.

